Introduction

If you love Ruby on Rails, you're going to love Docker. They are kindred spirits, born out of similar ideals.

For me, the allure of Rails was its Big Ideas: generators, migrations, testing as a first-class citizen, convention over configuration, multi-environment setups built in, live-code reloading. While, individually, these features may not have been new, the combination made Rails more than the sum of its parts: it gave us superpowers.

Docker is doing for DevOps what Rails did for web development. It too is packed with Big Ideas: a holistic view of your app (hint: your app is more than just your Rails code), containerization (lighter-weight, faster, and more efficient than VMs), software delivery that doesn't suck (for example, Ruby installs the first time you run a Ruby script), fault-tolerant clustering and scaling out of the box (spin up production-like clusters on your local machine), expert-level security features baked in (for example, automatic key rotation). The list goes on.

Docker is lowering the barrier to entry, making DevOps tasks that previously would have been unthinkable suddenly within our grasp. It gives us a *new* set of superpowers.

That said, Docker is not a panacea or a silver bullet to solve all your DevOps challenges. As with all technologies, there are trade-offs (I'll try to point these out as we go). However, despite the trade-offs, as you'll discover in this book, there is value in adopting Docker.

What Is Docker?

Docker, the technology, is a set of tools built around the idea of packaging and running software in small, sandboxed environments known as *containers* (we'll get to the nitty gritty of these in What Is a Container?, on page 6).